

Lab 3: Linearizability of Lock-free Skiplists

Dawa Arkhang, Zhengan Chen

October 2026

1 Measuring execution time

1.1 Measurement program

Distribution	Operation Mix	Threads	Avg Execution Time (ms)
Uniform	1:1:8	1	10.32
Uniform	1:1:8	2	17.27
Uniform	1:1:8	4	19.32
Uniform	1:1:8	8	31.42
Uniform	1:1:0	1	24.60
Uniform	1:1:0	2	27.31
Uniform	1:1:0	4	30.98
Uniform	1:1:0	8	54.09
Normal	1:1:8	1	14.33
Normal	1:1:8	2	18.53
Normal	1:1:8	4	23.15
Normal	1:1:8	8	36.73
Normal	1:1:0	1	21.80
Normal	1:1:0	2	27.24
Normal	1:1:0	4	32.75
Normal	1:1:0	8	61.14

Table 1: Task 1.1: Local baseline execution times across 10 measurement rounds.

1.2 PDC experiment

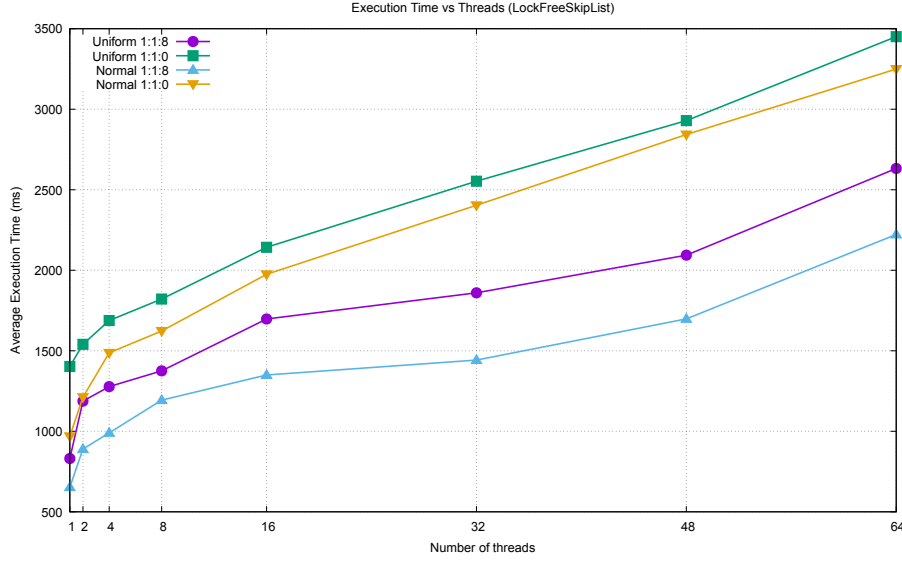


Figure 1: Execution time of the skip list under different thread number, operation ratios and value distributions.

The graph shows two clear trends. First, as the number of threads increases, the running time for the 1:1:8 ratio is lower than for the 1:1:0 ratio. Second, the Uniform distribution is faster than the Normal distribution.

This makes sense because the 1:1:8 ratio means 10% add, 10% remove, and 80% contains operations. A contains operation does not need CAS, so it usually has fewer retries and less contention. This advantage becomes more noticeable as the number of threads increases, because more threads also mean a higher chance of contention. The Normal distribution means that some values are accessed more frequently than others. This causes more operations to target the same area of the skip list, which increases contention between threads. Therefore, theoretically, the fastest case should be Uniform with a 1:1:8 ratio, while the slowest should be Normal with a 1:1:0 ratio. This is exactly what we observe in the figure.

2 Identify and validate linearization points

2.1 Identify linearization points

In a lock-free skiplist, the abstract state of the set is defined strictly by the keys linked and unmarked at the bottom level (level 0). Higher levels are shortcuts

designed to achieve $O(\log n)$ search efficiency. Operations that mutate or observe membership linearize either at an atomic operation targeting level 0 or during an atomic read inside `find()`.

Linearization Points in `add()`

```

1 Node<T> pred = preds[bottomLevel];
2 Node<T> succ = succs[bottomLevel];
3
4 if (!pred.next[bottomLevel].compareAndSet(succ, newNode, false, false)) {
5     continue;
6 }
```

Why this is the LP: Before this instruction executes, `newNode` exists only on the thread's local stack and is completely disconnected from the list. The exact moment this hardware CAS instruction succeeds on level 0, `newNode` is atomically linked into the shared structure reachable from `head`. The item instantly transitions from absent to present in the abstract set. Subsequent CAS operations that link higher levels are performance shortcuts, not linearization points.

Linearization Points in `remove()`

```

1 while (true) {
2
3     boolean iMarkedIt = nodeToRemove.next[bottomLevel].compareAndSet(succ, succ, false, true);
4     succ = succs[bottomLevel].next[bottomLevel].get(marked);
5     if (iMarkedIt) {
6         find(x, preds, succs);
7         return true;
8     }
9 }
```

Why this is the LP: Marking higher levels (from `topLevel` down to level 1) merely deters traversing threads from using shortcuts through the target node; it does not delete it. The decisive linearization point is the CAS instruction that sets the mark bit to `true` on level 0. Because abstract membership requires an unmarked node at level 0, this single atomic commit logically removes the key from the set. Physical unlinking happens later during traversal cleanup.

```

1 boolean found = find(x, preds, succs);
2 if (!found) {
3     return false;
4 }
5 ...
6 else if (marked[0]) return false;
```

Why this is the LP:

- **Key absent:** If the node does not exist, the operation linearizes inside `find()` at the bottom-level atomic read that confirms the target is not in the set.
- **Concurrent deletion:** If the node was found but another thread marked its bottom-level pointer first, this operation linearizes at the instant the competing thread successfully executed its level 0 marking CAS.

Linearization Points Inside `find()`

A function call spans multiple instructions and cannot be instantaneous on its own. For operations that depend on read-only results from `find()` (such as unsuccessful `add()` and absent `remove()`), the linearization point occurs inside the execution of `find()` at the bottom level:

```
1 for (int level = MAX_LEVEL; level >= bottomLevel; level--) {
2     curr = pred.next[level].getReference();
3     while (true) {
4         succ = curr.next[level].get(marked); // LP when level ==
5         bottomLevel
6         while (marked[0]) {
7             snip = pred.next[level].compareAndSet(curr, succ, false
8             , false);
9             if (!snip) continue retry;
10            curr = pred.next[level].getReference();
11            succ = curr.next[level].get(marked);
12        }
13        if (curr.value != null && x.compareTo(curr.value) < 0) {
14            pred = curr;
15            curr = succ;
16        } else {
17            break;
18        }
19        preds[level] = pred;
20        succs[level] = curr;
21    }
22    return (curr.value != null && x.compareTo(curr.value) == 0);
```

Why this is the LP: The linearization point occurs at the last execution of `succ = curr.next[level].get(marked)` at `bottomLevel` (level 0) before the loop exits and the return value is evaluated:

- **Duplicate found (unsuccessful add):** The atomic read witnesses that an unmarked node holding key `x` is actively present in the bottom-level chain at that instant, justifying the insertion rejection.
- **Node absent (unsuccessful remove):** The atomic read inspects the bottom-level neighborhood where `x` ought to reside, establishing that no unmarked node with key `x` exists in the set at that exact microsecond.

Linearization Points in contains()

```
1 for (int level = MAX_LEVEL; level >= bottomLevel; level--) {
2     curr = pred.next[level].getReference();
3     while (true) {
4         succ = curr.next[level].get(marked);
5         while (marked[0]) {
6             curr = pred.next[level].getReference();
7             succ = curr.next[level].get(marked);
8         }
9         if (curr.value != null && x.compareTo(curr.value) < 0) {
10             pred = curr;
11             curr = succ;
12         } else {
13             break;
14         }
15     }
16 }
17 return (curr.value != null && x.compareTo(curr.value) == 0);
```

Why this is the LP: contains() is read-only and lock-free. Its linearization point is the bottom-level atomic read `curr.next[bottomLevel].get(marked)`:

- If `curr.value == x` and `marked[0]` is false, the instantaneous read certifies that the item is a valid member of the abstract set.
- If the key is absent or its mark bit is true, that same atomic read certifies that the node is either logically deleted or not present at that instant.

Special Case: remove() when find() is True but iMarkedIt is False

A subtle edge case occurs when `find()` locates the node, but the subsequent bottom-level CAS returns false:

```
1 boolean iMarkedIt = nodeToRemove.next[bottomLevel].compareAndSet(
2     succ, succ, false, true);
3 succ = succs[bottomLevel].next[bottomLevel].get(marked);
4 if (iMarkedIt) {
5     find(x, preds, succs);
6     return true;
7 } else if (marked[0]) {
8     return false; // Found, but another thread marked it first
9 }
```

Location of the LP: As highlighted in the assignment notes, an operation's linearization point does not necessarily occur on the thread executing the operation. Here, the current thread's CAS failed because another concurrent thread executed a successful bottom-level marking CAS on `nodeToRemove` moments

earlier. The current thread’s unsuccessful `remove()` is linearized at the **exact moment the competing thread successfully marked the node at level 0**. The subsequent read `marked[0]` merely observes that this state transition has already occurred.

Techniques to Capture Linearization Points

To empirically validate linearizability, every invocation must record its start time, end time, and the exact timestamp of its linearization point ($t_{\text{start}} < t_{\text{LP}} < t_{\text{end}}$). Possible techniques to capture these points include:

- **Direct Sampling on Successful Self-Mutations:** For successful insertions and successful removals (`iMarkedIt == true`), the thread captures the timestamp immediately after its CAS succeeds.
- **Timestamp Stamping in Shared Nodes (for the `!iMarkedIt` Case):** Because the LP of an unsuccessful remove where `iMarkedIt == false` was performed by a competing thread, the reading thread cannot independently determine when the CAS occurred. A robust capture technique is to store a timestamp field inside the `Node` class. The thread that succeeds in marking the node records its timestamp into `node.markedTimestamp`; subsequent threads observing `marked[0] == true` can then retrieve the exact LP timestamp directly from the node.
- **Logging Architecture:** Captured records can be appended to a shared synchronized structure (global log) or written to thread-private memory buffers (local logs) that are merged and sorted post-execution to minimize cache interference during measurement.

2.2 Develop a validation method

The validation method models a ground-truth sequential execution against a standard sequential reference structure, `java.util.HashSet<Integer>`. The validation proceeds through three stages:

1. **Total Ordering by Timestamp:** The captured log entries are sorted in ascending order using their timestamps (`System.nanoTime()`). This chronologically reconstructs the alleged sequence of linearization points across all threads.
2. **Oracle Execution:** We iterate sequentially over the sorted trace. For each log entry, the corresponding method (`ADD`, `REMOVE`, or `CONTAINS`) is invoked directly on an initially empty `HashSet` instance with the recorded

key argument. The return value from the `HashSet` is stored as the expected outcome (`flag`).

3. **Discrepancy Evaluation:** The expected result (`flag`) produced by the oracle is directly compared against the return value recorded during concurrent execution (`e.ret`). `flag != e.ret` indicates that the concurrent skiplist behaved inconsistently with the sequential specification at that timestamp. We therefore incrementing the discrepancy counter.

2.2.1 Interpretation of Results

A discrepancy count of zero indicates that the total order corresponds to a valid sequential history. If the discrepancy count is strictly positive, it demonstrates either:

- A flaw in the data structure’s linearizability
- Measurement error introduced by the interleaving of other threads between the true linearization point and the invocation of `System.nanoTime()`.

2.3 Locked time sampling

We evaluated the locked time sampling implementation (`LockFreeSkiplistLocked`) locally across 1, 2, 4, and 8 threads executing 100 000 operations per thread, comparing it against the uninstrumented baseline (`Default`). The benchmark was performed across both Uniform and Normal value distributions for read-dominated (1:1:8) and write-only (1:1:0) workloads. The measured execution times and discrepancy scores from our validation method are presented in Table 2.

2.3.1 Performance Lag Analysis

The lock-protected implementation lags behind the default baseline substantially as thread count scales:

- **Single-threaded Baseline ($T = 1$):** With a single thread, the overhead is minimal. No lock contention exists so the minor lag is caused by the cost of uncontended lock acquire/release calls, querying `System.nanoTime()`, and allocating `Log.Entry` objects into the `ArrayList`.
- **High-thread Contention ($T = 4, 8$):** At 8 threads, the locked implementation exhibits severe degradation, slowing down by up to $4.85\times$ in read-heavy workloads (from 66.53 ms to 322.51 ms) and up to $3.10\times$ in write workloads. Because all operations must acquire the central `ReentrantLock`

Workload	Threads	Default (ms)	Locked (ms)	Slowdown	Discrepancies
Uniform (1:1:8)	1	24.02	27.73	1.15×	0.0
Uniform (1:1:8)	2	30.99	55.46	1.79×	0.0
Uniform (1:1:8)	4	40.78	201.72	4.95×	1.0
Uniform (1:1:8)	8	66.53	322.51	4.85×	1.8
Uniform (1:1:0)	1	43.74	45.53	1.04×	0.0
Uniform (1:1:0)	2	52.87	72.81	1.38×	0.0
Uniform (1:1:0)	4	68.30	185.92	2.72×	0.0
Uniform (1:1:0)	8	116.38	361.26	3.10×	0.0
Normal (1:1:8)	1	25.34	29.92	1.18×	0.0
Normal (1:1:8)	2	35.32	51.04	1.45×	0.0
Normal (1:1:8)	4	43.60	164.94	3.78×	1.6
Normal (1:1:8)	8	68.28	321.16	4.70×	9.8
Normal (1:1:0)	1	40.13	44.32	1.10×	0.0
Normal (1:1:0)	2	48.47	64.34	1.33×	0.0
Normal (1:1:0)	4	63.74	203.14	3.19×	0.0
Normal (1:1:0)	8	89.81	336.66	3.75×	0.0

Table 2: Comparison of execution times and validation discrepancies between Default and Locked skiplist.

to sample timestamps, threads become globally serialized, negating the concurrency advantages of the underlying lock-free skiplist.

2.3.2 Accuracy and Discrepancy Origins

The accuracy satisfies the passing requirement (< 100 discrepancies):

- For the write-only workload (1:1:0), the discrepancy count is **0.0** across all thread counts, showing that mutation linearization points were bounded accurately by the lock.
- In the read-heavy workload (1:1:8), minor discrepancies appear at 4 threads (≈ 1.0) and 8 threads (≈ 9.8 under Normal distribution).
- **Origin of Discrepancies:** These non-zero values stem from the inherent resolution limits of `System.nanoTime()` and timer interleaving across CPU cores. When multiple threads release and reacquire the lock in tight succession, consecutive operations on independent cores can obtain identical or sub-nanosecond reordered timestamps, causing `Arrays.sort` to interleave a read right before or after a state change. Under the Normal distribution, keys cluster tightly around the mean, intensifying access to identical cache lines and magnifying this timer jitter.

2.4 Lock-free time sampling with local log

To eliminate the global lock bottleneck identified in Task 2.3, we implemented `LockFreeSkipListLocal`.

- The data structure maintains an array of disjoint lists `List<Log.Entry>[] threadLog`. Each thread i appends records strictly to its dedicated `threadLog[threadId]` without acquiring locks or performing atomic operations on the collection.
- Time sampling occurs without mutual exclusion. Once an operation completes its linearization condition, the thread calls `System.nanoTime()` and appends the entry to its local log.
- Upon benchmark completion, `getLog()` aggregates all thread-local lists into a unified array. Chronological ordering is reconstructed by `Log.validate` via a timestamp sort.

2.4.1 Experimental Results and Comparative Analysis

Table 3 compares the execution time and discrepancy metrics across all three implementations (`Default`, `Locked`, and `LocalLog`) for local runs of 100 000 operations per thread across 1, 2, 4, and 8 threads.

Workload	Threads	Default (ms)	Locked (ms)	Local (ms)	Locked Disc.	Local Disc.
Uniform (1:1:8)	1	24.02	27.73	25.51	0.0	0.0
Uniform (1:1:8)	2	30.99	55.46	34.88	0.0	0.0
Uniform (1:1:8)	4	40.78	201.72	48.73	1.0	0.0
Uniform (1:1:8)	8	66.53	322.51	107.06	1.8	0.6
Uniform (1:1:0)	1	43.74	45.53	47.08	0.0	0.0
Uniform (1:1:0)	2	52.87	72.81	61.07	0.0	0.0
Uniform (1:1:0)	4	68.30	185.92	104.74	0.0	0.4
Uniform (1:1:0)	8	116.38	361.26	211.42	0.0	4.2
Normal (1:1:8)	1	25.34	29.92	26.69	0.0	0.0
Normal (1:1:8)	2	35.32	51.04	37.42	0.0	0.0
Normal (1:1:8)	4	43.60	164.94	50.02	1.6	0.0
Normal (1:1:8)	8	68.28	321.16	109.61	9.8	2.6
Normal (1:1:0)	1	40.13	44.32	45.14	0.0	0.0
Normal (1:1:0)	2	48.47	64.34	53.19	0.0	0.0
Normal (1:1:0)	4	63.74	203.14	70.86	0.0	0.0
Normal (1:1:0)	8	89.81	336.66	126.57	0.0	2.0

Table 3: Performance and discrepancy comparison between `Default`, `Locked`, and `LocalLog` time sampling.

Scalability and Performance Recovery: By removing the global lock, `LocalLog` recovers substantial multithreaded scalability:

- At 8 threads on read-heavy workloads (Uniform 1:1:8), execution time decreases from 322.51 ms under `Locked` down to 107.06 ms under `LocalLog` (a $\approx 3\times$ speedup).
- The overhead relative to the uninstrumented baseline (`Default`) is low, driven primarily by object allocations (`new Log.Entry`) and calls to `System.nanoTime()` rather than synchronization contention.

Discrepancy Analysis: Across all configurations, discrepancy counts remained between 0.0 and 4.2, satisfying the requirement (< 100).

- Because the gap between an atomic memory modification (e.g., CAS at level 0) and the timer read is unprotected, concurrent threads can interleave. For instance, thread T_1 can complete a successful `remove(x)`, but before T_1 samples `System.nanoTime()`, thread T_2 linearizes an unsuccessful `contains(x)` and samples the clock first.
- The subsequent sort places T_2 's read before T_1 's deletion in the sequential replay, producing a discrepancy in `HashSet` verification. As concurrency increases (4 to 8 threads), scheduling interleavings become more frequent, leading to slight increases in discrepancy counts.

2.5 Lock-free time sampling with global log

We implemented `LockFreeSkipListGlobal` using a shared `ConcurrentLinkedQueue<Log.Entry>`. Following each linearization check or CAS, the thread reads `System.nanoTime()` and appends the entry via `log.add(...)` without acquiring explicit locks. Once all threads complete, `getLog()` flattens the queue into an array.

2.5.1 Java Memory Model (JMM) and Happens-Before Analysis

Under the JMM, `ConcurrentLinkedQueue.add()` executes volatile reads and atomic `compareAndSet` operations on internal node references and the tail pointer.

- A successful CAS on the queue *synchronizes-with* subsequent volatile operations by other threads enqueueing to the tail, transitively making all prior memory writes (including preceding skiplist operations) visible across threads.
- In contrast to `LocalLog`, where logging writes are isolated to thread-local memory, sharing a single concurrent queue introduces cache-coherence traffic (MESI invalidations) around the queue's tail pointer, adding moderate contention at higher core counts.
- Because the time-sampling window between the skiplist linearization point and `System.nanoTime()` remains unprotected, race interleavings and discrepancies can still occur.

2.5.2 Results and Accuracy

As shown in Table 4, both implementations maintain high accuracy with discrepancy counts below 5.0 (well beneath the threshold of 100). **LocalLog** achieves slightly faster execution times than **GlobalLog** at 8 threads (e.g., 107.06 ms vs. 146.72 ms in Uniform 1:1:8) due to the complete absence of shared-memory contention during logging.

Workload	Threads	LocalLog (ms)	GlobalLog (ms)	Local Disc.	Global Disc.
Uniform (1:1:8)	1	25.51	25.65	0.0	0.0
Uniform (1:1:8)	2	34.88	39.02	0.0	0.0
Uniform (1:1:8)	4	48.73	56.19	0.0	0.0
Uniform (1:1:8)	8	107.06	146.72	0.6	0.2
Uniform (1:1:0)	1	47.08	47.75	0.0	0.0
Uniform (1:1:0)	2	61.07	70.16	0.0	0.0
Uniform (1:1:0)	4	104.74	106.96	0.4	0.0
Uniform (1:1:0)	8	211.42	194.13	4.2	2.2
Normal (1:1:8)	1	26.69	29.51	0.0	0.0
Normal (1:1:8)	2	37.42	42.09	0.0	0.0
Normal (1:1:8)	4	50.02	61.90	0.0	0.0
Normal (1:1:8)	8	109.61	137.75	2.6	1.2
Normal (1:1:0)	1	45.14	46.87	0.0	0.0
Normal (1:1:0)	2	53.19	58.76	0.0	0.0
Normal (1:1:0)	4	70.86	79.43	0.0	0.4
Normal (1:1:0)	8	126.57	176.24	2.0	2.0

Table 4: Performance and accuracy comparison between LocalLog and GlobalLog time sampling.

2.6 PDC experiments

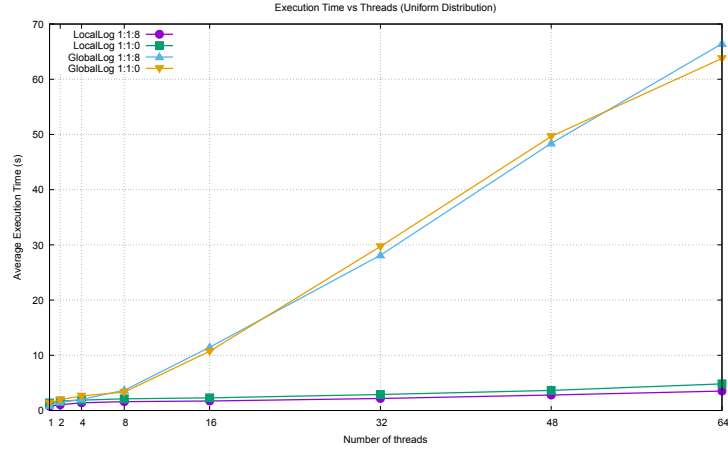


Figure 2: Execution time of the skip list under different thread number and operation ratios in uniform distributions.

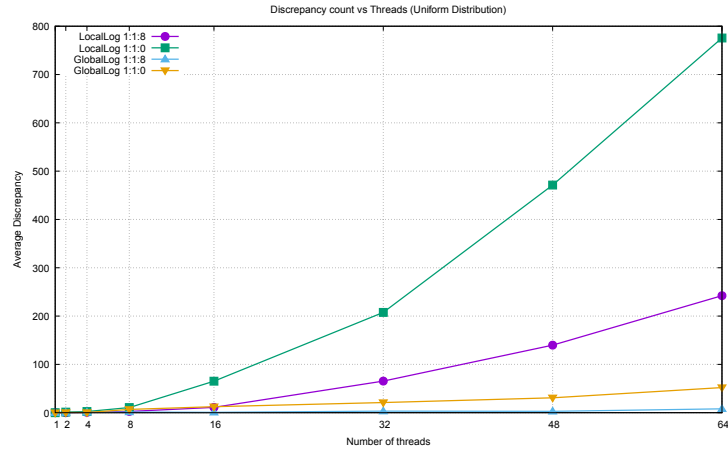


Figure 3: Discrepancy count of the skip list under different thread number and operation ratios in uniform distributions.

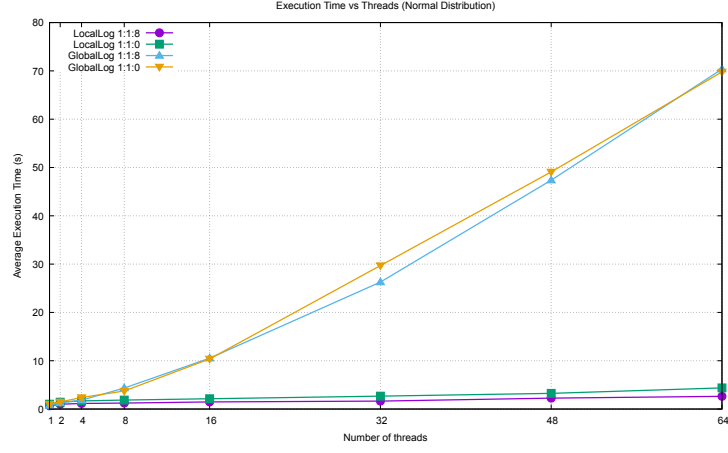


Figure 4: Execution time of the skip list under different thread number and operation ratios in normal distributions.

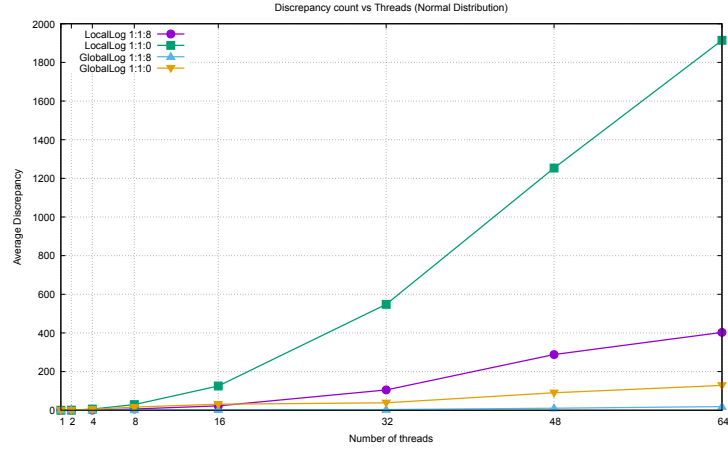


Figure 5: Discrepancy count of the skip list under different thread number and operation ratios in normal distributions.